

WSL / Docker シミュレータ表示トラブル - 会話ログ

Assistant

前提が重要に変わります。今の説明だと、以前の「WSLg/X11/WaylandでGUIウィンドウを出す」話は主因ではなく、**シミュレータと表示用プログラム間の通信問題**として見るべきです。

はい、**前のこちらの認識は一部ずれていました。**

前の回答では、主にこういう前提で話していました。

WSL / Docker コンテナ内のプログラムが、直接 Windows 側に GUI ウィンドウを出す
つまり WSLg / X11 / Wayland / OpenGL の問題

でも、今の説明だと実際はこうですね。

[Dockerコンテナ or WSL上のシミュレータ]
↓ TCP/UDP通信
[Windowsローカル上の表示用プログラム]

つまり、表示用プログラムは **別プロセス** で、シミュレータが出している通信データを **WSLのIPアドレス + ポート番号** で受け取って表示している。

この場合、主戦場は **GUI転送ではなくネットワーク通信** です。

正しい問題設定

今回見るべき構造はこれです。

Windows上の表示用プログラム
↓
WSLのIPアドレス:ポート番号
↓
WSL内のDockerポート公開
↓
コンテナ内のシミュレータ

なので、コンテナによって動く・動かないがあるなら、怪しいのは主にこれです。

項目	見るべきこと
Docker run の <code>-p</code>	ホスト側ポートとコンテナ側ポートが正しく対応しているか
TCP / UDP	プロトコルが合っているか
bind address	シミュレータが <code>127.0.0.1</code> ではなく <code>0.0.0.0</code> で待ち受けているか
container network	<code>bridge</code> / <code>host</code> / <code>custom network</code> の違い
コンテナ内ポート	実際にシミュレータがそのポートで listen しているか

項目	見るべきこと
WSL IP	表示用プログラムが指定している WSL IP が現在のものか
firewall	Windows 側でブロックされていないか
image差	シミュレータの設定ファイル、起動引数、依存ライブラリ、デフォルトポートが違わないか

前の話でまだ有効な部分

前の話のうち、これはまだ有効です。

同じPC上で、動くコンテナと動かないコンテナがあるなら、
コンテナイメージ差・docker run差を比較するべき

ただし、見るべき差分が変わります。

前は：

```
/mnt/wslg
/tmp/.X11-unix
DISPLAY
WAYLAND_DISPLAY
OpenGL
```

を見ていました。

でも今の構成なら、優先すべきは：

```
PortBindings
NetworkMode
待ち受けアドレス
TCP/UDP
コンテナ内の listen 状態
シミュレータの起動引数
表示用プログラムの接続先設定
```

です。

まず確認するべきこと

1. 動くコンテナと動かないコンテナのポート公開差分

```
docker ps
```

出力例：

CONTAINER ID	IMAGE	PORTS
abc123	sim-good	0.0.0.0:5000->5000/tcp
def456	sim-bad	5000/tcp

この場合、`bad` はコンテナ内でポートを持っていますが、WSL側に公開されていません。

さらに詳しく：

```
docker inspect good --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool
docker inspect bad --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool
```

見るべきはここです。

```
{
  "5000/tcp": [
    {
      "HostIp": "",
      "HostPort": "5000"
    }
  ]
}
```

2. TCP か UDP か

ここはかなり重要です。

もしシミュレータが UDP でデータを投げているのに、Docker 側で TCP として公開していると動きません。

TCP の場合：

```
docker run -p 5000:5000 image
```

UDP の場合：

```
docker run -p 5000:5000/udp image
```

両方必要なら：

```
docker run \
  -p 5000:5000/tcp \
  -p 5000:5000/udp \
  image
```

`docker ps` でこう見えるか確認します。

```
0.0.0.0:5000->5000/udp
```

3. コンテナ内でシミュレータが本当に待ち受けているか

動くコンテナ：

```
docker exec good ss -lntup
```

動かないコンテナ：

```
docker exec bad ss -lntup
```

TCPなら：

```
docker exec bad ss -lntp
```

UDPなら：

```
docker exec bad ss -lunp
```

見るべき例：

```
LISTEN 0 128 0.0.0.0:5000
```

これは良いです。

危ないのはこれ。

```
LISTEN 0 128 127.0.0.1:5000
```

これは **コンテナ内の localhost** でしか待ち受けていない状態です。

外から、つまり Windows の表示用プログラムからは届きません。

この場合、シミュレータの起動設定をこういう方向に変える必要があります。

```
--host 0.0.0.0
```

または config で：

```
host: 0.0.0.0  
port: 5000
```

今回の本命原因

君の状況だと、優先順位はこうです。

1. 片方の `docker run` に `-p` が足りない

動く方：

```
docker run -p 5000:5000 good-image
```

動かない方：

```
docker run bad-image
```

この場合、コンテナ内では動いていても、WSL外から見えません。

2. UDPなのに TCP として公開している

動く方：

```
docker run -p 5000:5000/udp good-image
```

動かない方：

```
docker run -p 5000:5000 bad-image
```

Docker の `-p 5000:5000` は基本 TCP 扱いです。

UDP が必要ななら `/udp` が必要です。

3. シミュレータが 127.0.0.1 に bind している

動く方：

```
0.0.0.0:5000
```

動かない方：

```
127.0.0.1:5000
```

この差はかなりありがちです。

表示用プログラムが WSL IP に接続するなら、シミュレータ側は基本的に：

```
0.0.0.0:ポート
```

で待ち受けている必要があります。

4. コンテナ内部のポート番号が違う

表示用プログラムと WSL 側は 5000 を見えても、コンテナ内シミュレータが 6000 で動いている可能性があります。

例えば：

```
docker run -p 5000:5000 bad-image
```

でもコンテナ内では：

```
0.0.0.0:6000
```

で待っている。

この場合は：

```
docker run -p 5000:6000 bad-image
```

が必要です。

形式は：

```
-p ホスト側ポート:コンテナ内ポート
```

です。

比較コマンド

動くコンテナを good、動かないコンテナを bad として、これを比較するとよいです。

```
for c in good bad; do
  echo "==== $c PORTS ====="
  docker port $c

  echo "==== $c NETWORK MODE ====="
  docker inspect $c --format '{{.HostConfig.NetworkMode}}'

  echo "==== $c PORT BINDINGS ====="
  docker inspect $c --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool

  echo "==== $c LISTEN ====="
  docker exec $c sh -lc 'ss -ltnp 2>/dev/null || netstat -ltnp 2>/dev/null'

  echo "==== $c ENV ====="
  docker exec $c sh -lc 'env | sort | grep -Ei "port|host|addr|ip|sim|display|server"'

  echo
done
```

ここで見るべき差は：

```
good は 0.0.0.0:5000->5000/udp になっている
bad は port binding がない

good は 0.0.0.0:5000 で listen
bad は 127.0.0.1:5000 で listen

good は 5000/udp
bad は 5000/tcp

good と bad で環境変数の SIM_PORT が違う
```

みたいなところです。

Windows 側から疎通確認

TCPなら PowerShell で：

```
Test-NetConnection <WSLのIPアドレス> -Port 5000
```

成功例：

```
TcpTestSucceeded : True
```

失敗例：

```
TcpTestSucceeded : False
```

ただし、これは **TCP専用** です。

UDP の場合はこれだけでは確認しにくいです。

UDPなら、WSL側・コンテナ側で `tcpdump` や `nc` を使って確認の方がよいです。

WSL IP の確認

WSL側で：

```
hostname -I
```

または：

```
ip addr show eth0
```

Windows の表示用プログラムの config に入れている IP が、この IP と一致しているか確認します。

ただし WSL2 の IP は、再起動で変わることがあります。

```
wsl --shutdown
```

をした後などに変わる可能性があります。

修正版の理解

今回の正しい理解はこうです。

問題は「コンテナのGUIがWindowsに出ない」ではない。

問題は、
Windowsローカルの表示用プログラムが、
WSL / Docker 内のシミュレータが出している通信ポートに

接続できていない、または正しいデータを受け取れていない
可能性が高い。

したがって、まず見るべきは：

1. Dockerの -p 設定
2. TCP/UDPの一致
3. コンテナ内の listen アドレス
4. コンテナ内ポートとホスト側ポートの対応
5. シミュレータの config / 起動引数
6. 表示用プログラムの config
7. WSL IP の現在値

です。

前の xeyes や DISPLAY の話は、シミュレータ自体がGUIを直接出す場合には有効ですが、今回の構成では優先度はかなり下がります。

User

試す必要のあるコマンドと、結果に対して何をすればいいかをまとめてください

Assistant

以下は、「同じPC・同じWSL・別Dockerコンテナで、表示できるものとできないものがある」前提の切り分け手順です。

以後、動くコンテナを good、動かないコンテナを bad、表示用ポートを仮に 5000 とします。

必要に応じて置き換えてください。

```
GOOD=good  
BAD=bad  
PORT=5000
```

0. まず全体像を確認

コマンド

```
docker ps
```

見るところ

PORTS 欄を見る。

例：


```
0.0.0.0:5000->5000/tcp
0.0.0.0:5000->5000/udp
```

結果ごとの対応

結果	判断	次にやること
good には 0.0.0.0:5000->... が あるが、bad にはない	bad はポート公開されていない	bad 起動時に -p を追加
good は /udp、bad は /tcp	TCP/UDP が違う	-p 5000:5000/udp にする
両方とも同じに見える	次の比較へ	docker inspect と ss を見る

1. Docker のポート公開設定を比較

コマンド

```
docker port $GOOD
docker port $BAD
```

さらに詳しく：

```
docker inspect $GOOD --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool
docker inspect $BAD --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool
```

良い例

```
{
  "5000/tcp": [
    {
      "HostIp": "",
      "HostPort": "5000"
    }
  ]
}
```

または UDP なら：

```
{
  "5000/udp": [
    {
      "HostIp": "",
      "HostPort": "5000"
    }
  ]
}
```

結果ごとの対応

結果	原因	対応
bad が {}	ポート公開されていない	<code>docker run -p 5000:5000 ...</code>
good が 5000/udp、bad が 5000/tcp	UDP/TCP 不一致	<code>docker run -p 5000:5000/udp ...</code>
ホスト側ポートが違う	表示プログラムが違うポートを見ている	表示プログラム config か <code>-p</code> を合わせる
コンテナ内ポートが違う	<code>-p</code> ホスト:コンテナ の右側が違う	例: コンテナ内が 6000 なら <code>-p 5000:6000</code>

2. NetworkMode を比較

コマンド

```
docker inspect $GOOD --format '{{.HostConfig.NetworkMode}}'
docker inspect $BAD --format '{{.HostConfig.NetworkMode}}'
```

結果ごとの対応

結果	意味	対応
両方 bridge	通常の Docker NAT	<code>-p</code> が必要
good が host、bad が bridge	ネットワーク方式が違う	bad も同じ方式にするか、 <code>-p</code> を正しく付ける
host	ホストネットワークを使う	<code>-p</code> は基本効かない。シミュレータの listen アドレスを見る
custom network	独自ネットワーク	まずは bridge + <code>-p</code> で再現確認した方がよい

3. コンテナ内で本当に待ち受けているか確認

TCP の場合

```
docker exec $GOOD sh -lc "ss -lntp || netstat -lntp"
docker exec $BAD sh -lc "ss -lntp || netstat -lntp"
```

UDP の場合

```
docker exec $GOOD sh -lc "ss -ltnp || netstat -ltnp"
docker exec $BAD sh -lc "ss -ltnp || netstat -ltnp"
```

良い例

TCP :

```
LISTEN 0 128 0.0.0.0:5000
```

UDP :

```
UNCONN 0 0 0.0.0.0:5000
```

悪い例

```
127.0.0.1:5000
```

結果ごとの対応

結果	原因	対応
bad で何も出ない	シミュレータがそのポートで起動していない	シミュレータの起動ログ・configを確認
bad が 127.0.0.1:5000	コンテナ内 localhost にしか bind していない	host: 0.0.0.0 に変える
bad が 0.0.0.0:6000	コンテナ内ポートが違う	-p 5000:6000 にする
good は UDP、bad は TCP	プロトコル違い	表示プログラム・シミュレータ・Docker のプロトコルを揃える
0.0.0.0:5000 で待っている	コンテナ内は OK	次は Docker port / Windows側疎通確認

4. シミュレータの起動引数・環境変数を比較

コマンド

```
docker inspect $GOOD --format '{{json .Config.Cmd}}' | python3 -m json.tool
docker inspect $BAD --format '{{json .Config.Cmd}}' | python3 -m json.tool

docker inspect $GOOD --format '{{json .Config.Entrypoint}}' | python3 -m json.tool
docker inspect $BAD --format '{{json .Config.Entrypoint}}' | python3 -m json.tool
```

環境変数 :

```
docker exec $GOOD sh -lc 'env | sort | grep -Ei "port|host|addr|ip|server|sim|display"'
docker exec $BAD sh -lc 'env | sort | grep -Ei "port|host|addr|ip|server|sim|display"'
```

結果ごとの対応

結果	原因	対応
SIM_PORT が違う	image / run 時の env 差	-e SIM_PORT=5000 など合わせる
HOST=127.0.0.1	外から届かない	HOST=0.0.0.0 にする
起動コマンドが違う	別モードで起動している	good と同じ起動引数に寄せる
config file path が違う	別設定を読んでいる	-v や config 指定を合わせる

5. WSL 側の IP を確認

WSL 側で実行

```
hostname -I
```

または：

```
ip addr show eth0
```

見るところ

表示用プログラムの config に書いている IP と一致しているか確認。

例：

```
172.28.123.45
```

表示用プログラム側も：

```
host: 172.28.123.45
port: 5000
```

になっている必要があります。

結果ごとの対応

結果	原因	対応
config の IP と hostname -I が違う	WSL IP が変わっている	表示用 config を更新
WSL 再起動後に変った	WSL2 の IP 変動	

結果	原因	対応
		起動ごとに確認するか、localhost forwarding を検討
IP は合っている	IPは主因ではなさそう	ポート・プロトコル・bind を見る

6. Windows 側から TCP 疎通確認

TCP の場合だけ有効です。PowerShell で実行。

```
Test-NetConnection <WSLのIPアドレス> -Port 5000
```

例：

```
Test-NetConnection 172.28.123.45 -Port 5000
```

結果ごとの対応

結果	意味	対応
TcpTestSucceeded : True	TCP接続はできている	表示プログラム側の設定・データ形式・プロトコルを疑う
TcpTestSucceeded : False	TCP接続できていない	-p、listen address、Windows Firewall、WSL IP を確認
UDP のシミュレータ	この確認は不十分	tcpdump / nc -u で確認

7. UDP かどうか確認

表示用プログラムとシミュレータが UDP を使っているなら、ここがかなり重要です。

Docker 側確認

```
docker ps
```

見るべき表示：

```
0.0.0.0:5000->5000/udp
```

UDP で公開する docker run

```
docker run -p 5000:5000/udp your-image
```

TCP と UDP 両方必要なら：

```
docker run \
  -p 5000:5000/tcp \
  -p 5000:5000/udp \
  your-image
```

結果ごとの対応

結果	原因	対応
5000/tcp しかない	UDPが公開されていない	-p 5000:5000/udp を追加
5000/udp はあるが表示されない	データが飛んでいない可能性	tcpdump でパケット確認
good と bad で TCP/UDP が違う	docker run 差	good に合わせる

8. パケットが来ているか確認

WSL 側で確認します。

tcpdump インストール

```
sudo apt update
sudo apt install -y tcpdump
```

TCP/UDP 両方見る

```
sudo tcpdump -i any port 5000
```

表示用プログラムを起動、または接続操作をしたときに、何か流れるか見る。

結果ごとの対応

結果	意味	対応
パケットが全く来ない	Windows表示プログラムからWSLに届いていない	IP、port、Firewall、表示プログラム configを確認
WSLには来ているがコンテナに届かない	Docker port mapping 問題	-p、TCP/UDP、NetworkMode を確認
コンテナまで届いているが表示されない	アプリケーション層の問題	

結果	意味	対応
		データ形式、バージョン、config差を確認

9. コンテナのログを見る

コマンド

```
docker logs $GOOD --tail=100
docker logs $BAD --tail=100
```

リアルタイムで見るなら：

```
docker logs -f $BAD
```

見るべき文字列

```
Listening on 0.0.0.0:5000
Listening on 127.0.0.1:5000
Port already in use
Connection refused
Bind failed
Using port 6000
UDP server started
TCP server started
Config loaded from ...
```

結果ごとの対応

ログ	意味	対応
Listening on 127.0.0.1	外から届かない	0.0.0.0 に変更
Port already in use	ポート競合	別ポートにするか既存プロセス停止
Using port 6000	想定と違うポート	-p 5000:6000 または config 修正
UDP server started	UDP	Dockerも /udp にする
エラーで落ちている	シミュレータ未起動	image依存・config・ライブラリ確認

10. まとめて比較するコマンド

まずこれを実行するとよいです。

```
GOOD=good
BAD=bad

for c in $GOOD $BAD; do
  echo "==== $c: docker ps ports ====="
  docker port $c || true

  echo "==== $c: network mode ====="
  docker inspect $c --format '{{.HostConfig.NetworkMode}}'

  echo "==== $c: port bindings ====="
  docker inspect $c --format '{{json .HostConfig.PortBindings}}' | python3 -m json.tool

  echo "==== $c: listening ports ====="
  docker exec $c sh -lc 'ss -ltnup 2>/dev/null || netstat -ltnup 2>/dev/null || true'

  echo "==== $c: relevant env ====="
  docker exec $c sh -lc 'env | sort | grep -Ei "port|host|addr|ip|server|sim|display" || true'

  echo "==== $c: command ====="
  docker inspect $c --format '{{json .Config.Entrypoint}} {{json .Config.Cmd}}'

  echo
done
```

典型パターン別の修正

パターンA： `bad` に port binding がない

状態

```
docker port bad
```

何も出ない。

修正

TCPなら：

```
docker run -p 5000:5000 your-image
```

UDPなら：

```
docker run -p 5000:5000/udp your-image
```

パターンB：UDPなのにTCPで公開している

状態

```
0.0.0.0:5000->5000/tcp
```

でも実際はUDP通信。

修正

```
docker run -p 5000:5000/udp your-image
```

両方出すなら：

```
docker run \  
-p 5000:5000/tcp \  
-p 5000:5000/udp \  
your-image
```

パターンC：コンテナ内で 127.0.0.1 に bind している

状態

```
LISTEN 0 128 127.0.0.1:5000
```

修正

シミュレータの config / 起動引数を変更。

例：

```
--host 0.0.0.0 --port 5000
```

または：

```
host: 0.0.0.0  
port: 5000
```

パターンD：コンテナ内ポートが違う

状態

```
bad の中では 0.0.0.0:6000  
表示用プログラムは WSL_IP:5000 を見ている
```

修正

```
docker run -p 5000:6000 your-image
```

意味は：

```
-p ホスト側ポート:コンテナ内ポート
```

パターンE：WSL IP が違う

状態

WSL：

```
hostname -I
```

が：

```
172.28.123.45
```

なのに、表示用プログラム config が：

```
172.28.100.20
```

になっている。

修正

表示用プログラムの config を現在の WSL IP に更新。

優先順位

まずはこの順番で見ればよいです。

1. docker ps で PORTS を見る
2. docker port good / bad を比較
3. ss -lntup でコンテナ内の待ち受けを見る
4. TCP/UDP が合っているか見る
5. 127.0.0.1 ではなく 0.0.0.0 で待っているか見る
6. WSL IP が表示用プログラムの config と合っているか見る
7. Windows 側から Test-NetConnection
8. tcpdump でパケット確認
9. docker logs でシミュレータの起動状態を見る

最もありがちな原因はこの3つです。

1. bad 側の docker run に -p が足りない
2. UDPなのに TCP として -p している
3. シミュレータが 127.0.0.1 に bind している